

Instanced Actors in UE 5.5

By : Saurabh Yadav

What are Instanced Actors?

- A new optimization system in UE 5.5 that reduces CPU & memory usage by replacing actors with instances.
- Designed for **dense open-world environments** where thousands of objects (trees, NPCs, props) exist.

(Instancing allows a single model to be rendered multiple times with minimal overhead.

For example, a scene filled with numerous trees can utilize this method effectively. Instead of loading each tree as a unique object, one base model can create many variations, thus conserving resources and decreases draw calls significantly.)

How Instanced Actors Work?

- Actors are replaced with Instanced Actors managed by InstancedActorsManager.
- Mass Entity System handle logic and data for these Instances.
- Hydration & Dehydration dynamically switch/converts between full actors and lightweight entities
- The conversion is controlled by the Mass LOD system using distance to viewer logic and physics traces.

Mass Entity System

- A high performance system for managing and processing large numbers of game objects by turning them into lightweight, efficiently managed Mass Entities.

Core Components:

Entities: Represents individual objects.

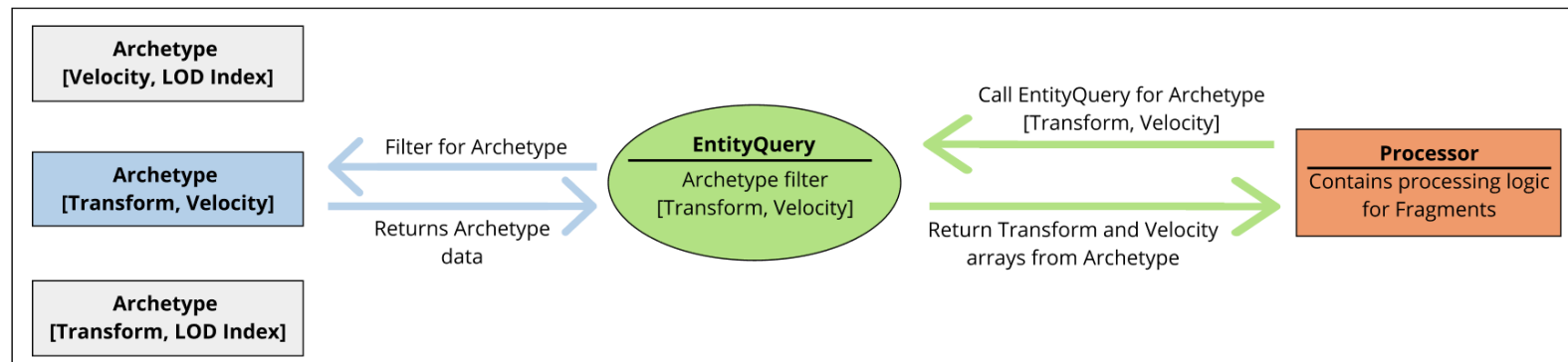
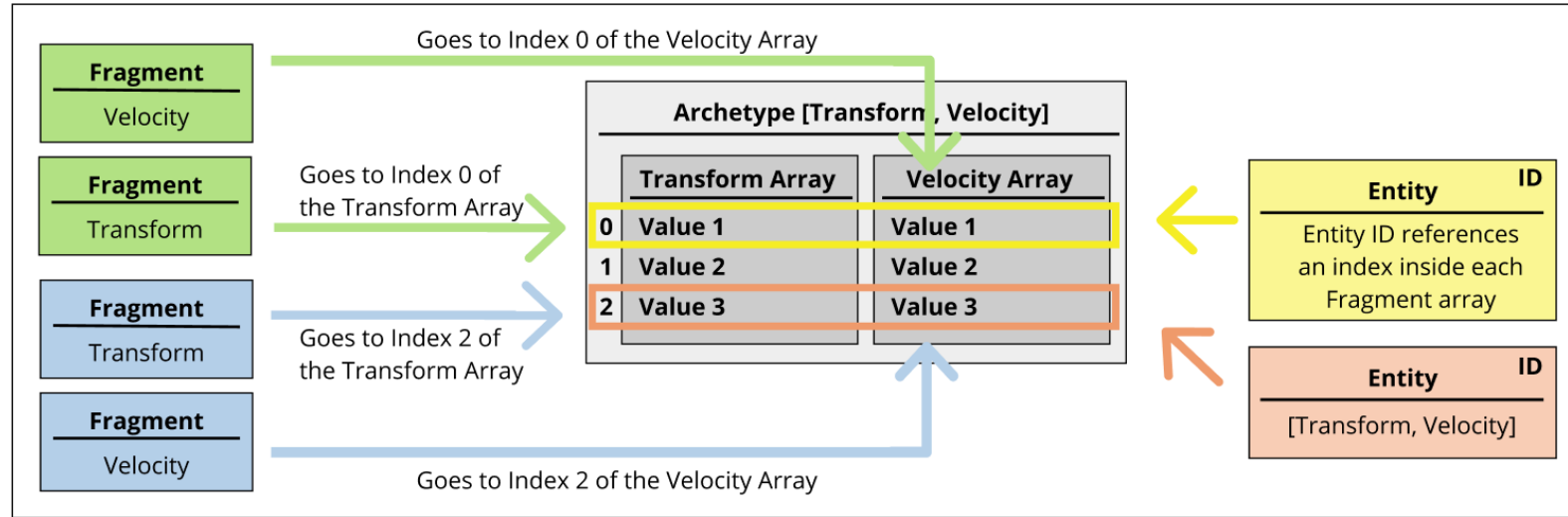
Fragments: atomic piece of data defining an entity's properties.

Archetype: Defines type of entities based on same fragment composition.

Chunks: Contiguous blocks of memory that stores entities of same archetype, improving data locality.

Processors: Functions that operates on collection of entities with specific fragment types, providing updates.

Mass Entity System



The Problem with Regular Actors:

- **Component Management:** Actors can have many components (mesh, collision, scripts, etc.), and the engine has to manage all of these for each Actor.
- **Tick Updates:** Even if an Actor isn't doing anything particularly complex, the engine still has to consider it during the tick (frame update) loop, checking for potential updates or events.
- **Rendering Overhead:** Each Actor typically results in a draw call, which is an instruction to the graphics card to draw something. With thousands of Actors, you get thousands of draw calls, which becomes a major performance bottleneck.
- **Memory Usage:** Each Actor consumes memory to store its components, properties, and other data.

How Mass Entities Solve This:

- **Data-Oriented Approach:** Instead of objects having a complex structure with components and individual logic, the Mass system stores only the essential data (fragments) needed for processing.
- **Efficient Data Storage:** Fragments of the same type (e.g., all the positions of the trees) are stored together in contiguous memory blocks (chunks). This significantly improves memory access and cache utilization, making processing much faster.
- **Parallel Processing:** The Mass system is designed for parallel processing. Processors can operate on chunks of entities much more efficiently than iterating through individual Actors.
- **Reduced Overhead:** Mass Entities have a much lower per-entity overhead compared to regular Actors. The engine doesn't have to manage complex components or perform individual tick updates for each entity, especially when represented as ISMs.



Hydration & Dehydration – Dynamic Actor Conversion

- Hydration: Converts Mass Entities → back into full actors when needed.
- Dehydration: Converts full actors → back into Mass Entities to save resources.
- Controlled by Mass LOD system (based on player distance & interactions).



Hydration & Dehydration – Dynamic Actor Conversion

- At game start, 100,000 trees exist in the world.
- Only 500 trees near the player are hydrated (full actors).
- Other 99,500 trees are dehydrated (converted to ISMs & Mass Entities).
- When the player moves closer, trees dynamically hydrate.
- Trees hit by a bullet or axe instantly hydrate to support physics destruction.

Use Cases

-  1. Open-World Games – Forests, cities, NPCs, vehicles.
-  2. Large Environments – Foliage, rocks, destructible objects.
-  3. AI Crowd Systems – Thousands of NPCs with efficient processing.
-  4. Traffic Systems – Cars are only fully active near the player.
-  5. Modular Level Design – Large city buildings, props, debris.